

CY IUT – GEII Neuville

Rapport Technique-Sous systèmes (IHM)

Projet Chef d'Orchestre Électronique

Document rédigé par : « Yigit-El karani »

Version : 1.0 – 10/04/2025

Avant-propos¹

Dans le cadre de notre projet d'orchestration électronique réalisé en troisième année de BUT GEII, notre équipe a eu pour mission de développer une interface permettant de créer, modifier et visualiser des sons en temps réel.

Pour cela, nous avons choisi d'utiliser Python, un langage simple à utiliser mais très puissant, notamment grâce à des bibliothèques comme numpy, scipy, matplotlib et sounddevice. Ce choix nous a permis de nous concentrer sur la logique du traitement du signal et sur l'interface, sans perdre de temps sur des problèmes techniques complexes.

Le système fonctionne sur un Raspberry Pi 4, un petit ordinateur très pratique pour embarquer notre application dans un boîtier musical. Grâce à ça, on peut envisager une utilisation autonome du dispositif, sans avoir besoin d'un PC externe.

Ce rapport présente les différentes étapes de conception de notre IHM, les choix que nous avons faits, et la manière dont notre module interagit avec les autres parties de l'orchestration.

Table des matières

Avant-propos	2
Table des matières	2-3
1. Introduction	4
2. Conception du système IHM	5
2.1 Objectifs de conception	5
2.2 Choix techniques	5
2.3 Architecture logicielle	6
3. Fabrication	6
3.1 Ecriture du code	6
3.2 Interface utilisateur	6
3.3 Visualisation graphique	6
3.4 Tests initiaux	7
3.5 Difficultés rencontrés	7
3.6 Etat final	7
3.7 Installation et mise en oeuvre sur Raspberry PI	7
4. Vérification et Tests	9
4.1 Procédures de tests	9
4.2 Procédure de test du système global	9
4.3 Liste des actions correctives	9
4.4 CHECKLIST du bon fonctionnement	10
5. Maintenance corrective et préventive	11
5.1 Plan de maintenance préventive globale	11

¹

5.2 Procédures de maintenance correctives	11
5.3 Fiche type de maintenance corrective	12
6. Implantation du système	13
6.1 Procédure de mise en oeuvre du système	13
6.2 Checklist de mise en oeuvre	13
6.3 Ouverture et améliorations possibles	14
7. Conclusion	14
Annexes	14

1. Introduction

Ce rapport technique présente le développement de notre partie du projet d'orchestration électronique, réalisé dans le cadre de la 3e année du BUT GEII. L'orchestration, en tant que système automatisé composé de plusieurs instruments pilotés numériquement, nécessite une coordination efficace entre plusieurs sous-systèmes. Notre groupe s'est chargé de la réalisation de l'interface homme-machine (IHM), un élément central permettant à l'utilisateur de générer, visualiser et modifier des signaux audio en temps réel.

L'IHM développée s'intègre dans l'architecture globale de l'orchestration. Elle permet :

- La génération de signaux MIDI bruités et numériques simulés (ondes sinusoïdales, carrées, etc.),
- L'application de traitements numériques : filtres passe-bas, passe-haut, passe-bande, inversion, décalage,
- La visualisation des signaux sur une interface graphique temps réel,
- La sortie sonore du signal traité, envoyée vers un système de diffusion ou d'enregistrement.

Le système repose sur une Raspberry Pi 4, qui exécute un programme Python utilisant des bibliothèques scientifiques (numpy, scipy, sounddevice, matplotlib) pour assurer l'ensemble des fonctions. L'IHM permet également une interaction clavier pour appliquer dynamiquement différents filtres au signal audio.

2. Conception du système IHM

2.1 Objectifs de conception

L'objectif principal de cette phase de conception était de réaliser une IHM capable de :

- Générer plusieurs signaux audio,
- Appliquer différents types de filtres numériques (passe-bas, passe-haut, passe-bande, coupe-bande, inversion, décalage),
- Visualiser en temps réel les signaux (individuels et combinés),
- Permettre une interaction simple avec l'utilisateur (commande par clavier).

Le tout devait fonctionner sur un système embarqué (Raspberry Pi 4) avec des performances suffisantes pour l'audio temps réel.

2.2 Choix techniques

- Langage de programmation : Python 3, pour sa simplicité, ses bibliothèques scientifiques, et sa compatibilité avec Raspberry Pi.
- Environnement de développement : Visual Studio Code (VS Code), utilisé pour l'écriture, le test et le débogage du code. Il offre une interface claire, des extensions utiles et une bonne compatibilité avec Raspberry Pi via SSH ou transfert de fichiers.
- Bibliothèques principales :

numpy : génération de signaux et traitement numérique,

scipy.signal : conception et application des filtres (Butterworth),

sounddevice : sortie audio en temps réel (interface avec la carte son),

matplotlib : visualisation graphique dynamique avec FuncAnimation.

- Fréquence d'échantillonnage : 44 100 Hz (standard audio, bonne qualité sans surcharge CPU).
- Raspberry Pi 4 : carte embarquée légère, compacte, avec sorties audio et GPIO.

2.3 Architecture logicielle

Le système se divise en plusieurs modules :

- Génération de signaux : fonctions spécifiques pour chaque type d'onde (MIDI bruité, carré, pseudo-carré).
- Filtrage : ensemble de fonctions qui appliquent un filtre choisi par l'utilisateur (commande via le clavier).
- Visualisation : mise à jour en temps réel de 5 courbes synchronisées (4 signaux + leur moyenne).
- Sortie audio : envoi du signal audio traité vers la sortie son de la Raspberry Pi.

3. Fabrication

3.1 Écriture du code

Le code Python a été structuré autour d'une animation matplotlib qui appelle la fonction `mettre_a_jour()` à chaque trame :

- À chaque appel, les 4 signaux sont générés (fréquences prédéfinies : 440 Hz, 523.25 Hz, 220 Hz, 110 Hz).
- Un filtre est sélectionné selon la touche clavier pressée (de 1 à 7).
- Le signal combiné est calculé comme la moyenne des 4 signaux filtrés.
- Le signal MIDI 1 est normalisé puis joué en sortie audio via `sounddevice.write()`.
- Les graphiques sont mis à jour dynamiquement avec les nouvelles données.

Le développement s'est fait dans Visual Studio Code (VS Code), configuré avec l'extension Python, l'autocomplétion, la coloration syntaxique, le débogueur intégré.

3.2 Interface utilisateur

- Interaction simple : l'utilisateur change le filtre appliqué aux signaux avec une touche (1 à 7).
- Pas de boutons : tout se fait au clavier pour plus de légèreté et de rapidité.
- Le titre des graphes indique dynamiquement le filtre en cours ("Original", "Passe-bas", etc.).

3.3 Visualisation graphique

5 sous-graphes sont affichés simultanément :

MIDI bruité 1 (440 Hz); MIDI bruité 2 (523.25 Hz); Numérique 1 (carré parasité 220 Hz), Numérique 2 (pseudo-carré 110 Hz); Signal combiné (moyenne des 4)

Les courbes sont mises à jour en temps réel avec `matplotlib.animation.FuncAnimation`, et nous fixons la limite d'amplitude à `[-320, 320]`, échantillons affichés = `TAILLE_TRAME`.

3.4 Tests initiaux

- Vérification que chaque signal était bien généré indépendamment.
- Application correcte des filtres : testés un par un avec les touches.
- Affichage cohérent des signaux dans les graphes.
- Lecture audio fluide, sans latence ni saturation.

3.5 Difficultés rencontrées

Besoin de synchroniser les signaux et le graphique en temps réel (nécessité de calibrer la taille des trames).

3.6 État final

À l'issue de la phase de fabrication, le système est fonctionnel, stable et prêt pour une démonstration.

3.8 Installation et mise en œuvre sur Raspberry Pi

3.8.1 Installation de la Raspberry Pi

- Utilisation d'une Raspberry Pi 4
- Connexion à un écran HDMI, clavier et souris
- Connexion au réseau local (Ethernet ou Wi-Fi)

3.8.2 Branchement

- Alimentation via port USB-C 5V / 3A
- Sortie audio utilisée : jack 3.5 mm
- Optionnel : ventilation passive ou active en cas de température élevée

3.8.3 Déploiement du programme

- Transfert du code Python depuis VS Code

3.8.4 Bon fonctionnement attendu

- Démarrage fluide de l'interface graphique sur écran
- Affichage des signaux à l'écran dans les 2 secondes
- Changement dynamique de filtre avec les touches 1 à 7 sans bug
- Stabilité confirmée après plusieurs minutes de fonctionnement

3.8.5 Mise à jour de Visual Studio Code

Mise à jour de VS Code recommandée régulièrement pour :

- Profiter des dernières corrections de bugs
- Bénéficier d'un meilleur support Python
- Assurer la compatibilité avec Raspberry Pi OS

4. Vérification et Tests

4.1 Procédures de tests

Pour valider le bon fonctionnement de l'IHM, plusieurs séries de tests ont été réalisées :

- Tests unitaires : vérification individuelle des fonctions de génération et de filtrage des signaux.
- Tests d'affichage : cohérence entre les signaux affichés et les signaux générés.
- Tests d'interaction : changement dynamique de filtre à l'aide des touches clavier.
- Tests de performance : évaluation de la consommation CPU et de la stabilité du flux audio.

Tous les tests ont été effectués via VS Code, permettant l'exécution directe du script Python, l'observation du terminal et la mise en évidence d'éventuelles erreurs en temps réel.

4.2 Procédure de test du système global

1. Lancement du script Python sur Raspberry Pi.
2. Vérification de l'ouverture correcte de la fenêtre matplotlib.
3. Attente du premier affichage des 5 signaux.
4. Pression successive sur les touches 1 à 7 pour tester tous les filtres.
5. Observation de la modification dynamique des signaux.
6. Fermeture propre du programme.

4.3 Liste des actions correctives

Problème rencontré	Solution apportée
Affichage ralenti avec les 5 graphiques	Réduction de la taille des trames, animation plus légère
Programme bloqué si une mauvaise touche est appuyée	Filtrage des touches acceptées dans la fonction clavier
Aucun affichage	Programme mal compilé

4.4 CHECKLIST du bon fonctionnement

Élément à vérifier	Statut	Détail
Raspberry bien alimenté	OK	
Ouverture correcte de l'interface graphique	OK	La fenêtre matplotlib s'ouvre sans erreur
Affichage des 5 courbes	OK	Tous les signaux s'affichent dès le lancement
Changement de filtre avec les touches clavier	OK	Touche 1 à 7 réagit correctement
Titre des graphes mis à jour	OK	Le nom du filtre appliqué est affiché
Stabilité générale de l'interface	OK	Pas de blocage après plusieurs minutes
Fermeture propre du programme	OK	La fenêtre se ferme sans erreur, le script se termine bien

5. Maintenance corrective et préventive

5.1 Plan de maintenance préventive globale

Afin d'assurer le bon fonctionnement sur le long terme, voici les actions de maintenance préventive à appliquer régulièrement :

Action	Fréquence recommandée	Objectif
Vérifier les mises à jour de Python	Toutes les semaines	Éviter les incompatibilités
Mettre à jour les bibliothèques Python	Tous les mois	Corriger bugs et failles
Mettre à jour VS Code	À chaque version stable	Assurer un environnement fiable
Nettoyer les fichiers inutiles	Tous les 2 mois	Alléger la charge système
Vérifier le bon fonctionnement des filtres	À chaque nouvelle session	Garantir la validité des résultats
Redémarrer le système proprement	1 fois par semaine	Prévenir les fuites mémoire

5.2 Procédures de maintenance correctives

En cas de dysfonctionnement ou bug constaté, voici les étapes recommandées :

1. Identifier clairement le symptôme (ex. : l'IHM ne s'ouvre pas, le clavier ne répond plus, une courbe ne s'affiche plus...)
2. Reproduire le problème pour confirmer qu'il est récurrent
3. Vérifier les erreurs affichées dans le terminal
4. Analyser le code à l'endroit concerné (souvent une mauvaise variable, une touche non prise en compte, etc.)
5. Corriger le bug, tester localement dans VS Code
6. Re-tester tout le fonctionnement global avant de considérer le bug résolu

5.3 Fiche type de maintenance corrective

Voici un modèle de fiche à remplir pour chaque intervention corrective :

Fiche de maintenance corrective	
Date	
Intervenant	
Problème	
Analyse effectuée	
Actions correctives appliqués	
Test	
Fonctionnement après les tests	

6. Implantation du système

6.1 Procédure de mise en œuvre du système

Pour mettre en œuvre le système IHM de traitement audio temps réel sur Raspberry Pi, voici les étapes à suivre :

1. Préparer la Raspberry Pi :
 - Installer Raspberry Pi OS
 - Mettre à jour le système avec `sudo apt update && sudo apt upgrade`
2. Installer les dépendances Python :
`numpy scipy matplotlib sounddevice`
3. Compiler le code Python sur la Raspberry Pi via VS
4. Connecter la sortie audio jack à un oscilloscope
5. Interagir avec l'IHM via le clavier (touches 1 à 7 pour les filtres)
6. Observer l'affichage graphique en temps réel des courbes sur l'écran relié à la Raspberry Pi et sur l'oscilloscope
7. Arrêter proprement le système en fermant la fenêtre ou en coupant le script

6.2 Checklist de mise en œuvre

Étape	Statut
Raspberry Pi alimentée et connectée	
Système d'exploitation fonctionnel	
Bibliothèques Python installées	
Programme lancé sans erreur	
Interface graphique visible	
Commandes clavier fonctionnelles	
Signaux affichés correctement	
Fermeture propre du programme	

6.3 Ouverture et améliorations possibles

Le système IHM actuel est stable et opérationnel, mais plusieurs évolutions peuvent être envisagées pour enrichir le projet :

- Enregistrement des signaux : sauvegarde des données générées ou filtrées au format
- Connexion à un orchestrion réel via GPIO ou MIDI : déclenchement physique de notes ou mouvements mécaniques
- Affichage d'un spectrogramme pour une meilleure analyse fréquentielle
- Contrôle à distance via une application web : serveur léger pour interagir depuis un autre appareil

7. Conclusion

Ce document devrait vous permettre a réalisé de A à Z, ce sous-système du projet d'orchestrion qui est la réalisation d'une IHM.

Ce projet a permis la conception et l'implémentation d'une IHM fonctionnelle et interactive dédiée à un orchestrion électronique. Le système développé en Python, compatible avec la Raspberry Pi, permet la génération de signaux audio, leur traitement numérique à l'aide de filtres, et leur visualisation en temps réel.

Annexes

Code : fichier midi4.py

Cahier des charges:

https://github.com/A-s-a-d/Table-de-Mixage/blob/Main/SousSysteme_IHM/CDC.pdf